

INFORME- SISTEMA DE VOZ CIUDADANA



DESARROLLO DE SOFTWARE – PC3

Nombre: Sánchez Longa Erick Joel 20221221D

Profesor: Manuel Alejandro Quispe torres

2026

ÍNDICE

1. Requisitos Funcionales (RF)	3
2. Historias de Usuario (HU)	4
2.1 HU-01	4
2.2 HU-02	4
2.3 HU-03	4
2.4 HU-04	4
2.5 HU-05	5
2.6 HU-06	5
2.7 HU-07	5
2.8 HU-08	5
3. Casos de Prueba (QA)	5
3.1 CP-01	5
3.2 CP-02	6
3.3 CP-03	7
3.4 CP-04	7
3.5 CP-05	9
4. Patrones de diseño	10
4.1 Facade	10
4.2 Decorator	10
4.3 Proxy	11
4.4. Composite	11
4.5 Flyweight	12

1. Requisitos Funcionales (RF)

ID	RF - 01
Nombre	Permitir la creación de una nueva propuesta legislativa.
Descripción	Permitir la creación de una nueva propuesta legislativa.
Actor Principal	Ciudadano
Prioridad	Alta

ID	RF - 02
Nombre	Listar Iniciativas
Descripción	Visualizar el catálogo de iniciativas activas y congeladas.
Actor Principal	Ciudadano
Prioridad	Alta

ID	RF - 03
Nombre	Registrar Firmas
Descripción	Sumar firmas de apoyo verificadas a una propuesta.
Actor Principal	Ciudadano
Prioridad	Alta

ID	RF - 04
Nombre	Comentar Iniciativa
Descripción	Añadir opiniones o sugerencias a una propuesta.
Actor Principal	Ciudadano
Prioridad	Media

ID	RF - 05
Nombre	Adjuntar Recursos
Descripción	Subir documentos que respalden la propuesta.
Actor Principal	Ciudadano
Prioridad	Media

ID	RF - 06
Nombre	Modificar Iniciativa
Descripción	Editar el contenido mientras no esté congelada
Actor Principal	Ciudadano
Prioridad	Alta

ID	RF - 07
Nombre	Congelar Iniciativa
Descripción	Bloquear edición al alcanzar el límite de firmas.
Actor Principal	Sistema
Prioridad	Alta

ID	RF - 08
Nombre	Derivar la propuesta una vez validada y congelada.
Descripción	Bloquear edición al alcanzar el límite de firmas.
Actor Principal	Sistema
Prioridad	Alta

2. Historias de Usuario (HU)

2.1 HU-01

Historia de usuario: Como ciudadano, quiero registrar una iniciativa para que sea apoyada.

Criterio de aceptación:

1. Debe requerir título y contenido.
2. El estado inicial debe ser "Activa".

2.2 HU-02

Historia de usuario: Como ciudadano, quiero ver las iniciativas para decidir a cuál apoyar.

Criterio de aceptación:

1. Mostrar título, firmas actuales y estado.
2. Ordenadas por fecha.

2.3 HU-03

Historia de usuario: Como ciudadano, quiero firmar una iniciativa para ayudar a su aprobación.

Criterio de aceptación:

1. Incrementar el contador en 1.
2. Validar que la firma sea de un proveedor.

2.4 HU-04

Como ciudadano, quiero comentar una iniciativa para aportar ideas.

Criterio de aceptación:

1. El comentario debe ser visible en el detalle.

2. No se permiten comentarios vacíos.

2.5 HU-05

Historia de usuario: Como ciudadano, quiero adjuntar recursos para respaldar la propuesta.

Criterio de aceptación:

1. Registrar el nombre del recurso asociado.

2.6 HU-06

Historia de usuario: Como autor, quiero modificar la iniciativa para corregir detalles.

Criterio de aceptación:

1. Solo se permite si el estado es "Activa".
2. Guardar los nuevos cambios.

2.7 HU-07

Historia de usuario: Como sistema, quiero congelar la iniciativa al llegar a la meta de firmas.

Criterio de aceptación:

1. Al llegar a N firmas, el estado cambia a "Congelada".
2. Se rechazan modificaciones.

2.8 HU-08

Historia de usuario: Como sistema, quiero enviar la iniciativa al Congreso tras congelarse.

Criterio de aceptación:

1. Disparar notificación de envío.
2. Cambiar estado a "Enviada".

3. Casos de Prueba (QA)

3.1 CP-01

Objetivo: Validar la creación exitosa de una iniciativa.

1. Ingresamos el título y la justificación

VOZ DEL CIUDADANO

Plataforma de Iniciativas Legislativas

Crear Iniciativa

Registrar Iniciativa

2. Se registra en la BD y se muestra activa

```
_id: ObjectId('6a22f4c2aaca5e65af2a9b18')
id: "071116a7"
titulo: "Aumento de Areas verdes en el distrito de ATE "
contenido: "Capítulo: Disposiciones Generales
          - Artículo: El distrito de ATE cue..."
firmas: 0
estado: "Activa"
comentarios: Array (empty)
adjuntos: Array (empty)
```

Resultado: Se puede ver como en MongoDB se muestra que la iniciativa esta: Activa además las firmas están en 0.

3.2 CP-02

Objetivo: Verificar que la lista muestra los datos reales de la BD.

Aumento de Areas verdes en el distrito de ATE

Activa

Capítulo: Disposiciones Generales
- Artículo: El distrito de ATE cuenta con reducidas zonas de areas verdes

Firmas: 0 / 5

Documentos Soporte: Ninguno

Comentarios: Ninguno

Firmar

Comentar

Choose File No file chosen

Subir Recurso

Enviar a Congreso

```
_id: ObjectId('6a22f4c2aaca!
id: "071116a7"
titulo: "Aumento de Areas v
contenido: "Capítulo: Dispo
          - Artículo: El
firmas: 0
estado: "Activa"
comentarios: Array (empty)
adjuntos: Array (empty)
```

Resultado: Se puede ver que los campos de la aplicación se corresponden a la de la base de datos

3.3 CP-03

Objetivo: Validar la adición dinámica de comentarios.

1. Escribimos un comentario, por ejemplo : Apoyo total.

Como se puede ver al darle clic a comentar, se guardo el comentario:

The screenshot shows a web interface for adding a comment. At the top, the title "Aumento de Areas verdes en el distrito de ATE" is displayed in red, with a red "Activa" button to its right. Below the title, the document details are shown: "Capítulo: Disposiciones Generales" and "- Artículo: El distrito de ATE cuenta con reducidas zonas de areas verdes". The status "Firmas: 0 / 5" is shown. Below this, "Documentos Soporte: Ninguno" is displayed. The "Comentarios:" section shows a red box around the text "Apoyo total". Below the comments section, there are two buttons: "Firmar" and "Comentar". The "Comentar" button is highlighted. To the right of the "Comentar" button is a text input field with the placeholder "Escribe un comentario...". Below the input field is a "Choose File" button and a "No file chosen" label. At the bottom of the form, there are two large buttons: "Subir Recurso" (orange) and "Enviar a Congreso" (purple).

2. El array comentarios se actualiza en MongoDB.

```
▶ _id: ObjectId('6a22f4c2aaca5e65af2a9b18')
  id: "071116a7"
  titulo: "Aumento de Areas verdes en el distrito de ATE "
  contenido: "Capítulo: Disposiciones Generales
             - Artículo: El distrito de ATE cue..."
  firmas: 0
  estado: "Activa"
  ▼ comentarios: Array (1)
    0: "Apoyo total "
  ▶ adjuntos: Array (empty)
```

3.4 CP-04

Objetivo: Validar subida física y enlace de recurso.

1. Seleccionar un archivo PDF en el input file.

Aumento de Areas verdes en el distrito de ATE
Activa

Capítulo: Disposiciones Generales
- Artículo: El distrito de ATE cuenta con reducidas zonas de areas verdes

Firmas: 0 / 5

Documentos Soporte: Ninguno

Comentarios: Apoyo total

Firmar

Comentar

Choose File

Documentacion.pdf

Subir Recurso

Enviar a Congreso

Luego de subir el archivo

Aumento de Areas verdes en el distrito de ATE
Activa

Capítulo: Disposiciones Generales
- Artículo: El distrito de ATE cuenta con reducidas zonas de areas verdes

Firmas: 0 / 5

Documentos Soporte:

Documentacion.pdf

Comentarios: Apoyo total

Firmar

Comentar

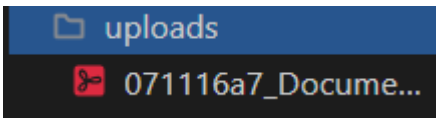
Choose File

No file chosen

Subir Recurso

Enviar a Congreso

2. El PDF se guarda localmente. En MongoDB se guarda la ruta

LOCALMENTE	MONGODB
	<pre> { "_id": ObjectId("6a22f4c2aaca5e65af2a9b18"), "id": "071116a7", "titulo": "Aumento de Areas verdes en el distrito de ATE ", "contenido": "Capítulo: Disposiciones Generales - Artículo: El distrito de ATE cue...", "firmas": 0, "estado": "Activa", "comentarios": Array (1), "adjuntos": Array (1) 0: "uploads/071116a7_Documentacion.pdf" } </pre>

Resultado: El PDF se guarda localmente. En MongoDB se guarda la ruta (uploads/ID_archivo.pdf). Aparece un enlace rojo clickeable en la interfaz para poder descargar el archivo.

3.5 CP-05

Objetivo: Validar transición automática a “congelada”.

1. Clic en "Firmar" 5 veces para activar “congelada”.

Aumento de Areas verdes en el distrito de ATE

Congelada

Capítulo: Disposiciones Generales
- Artículo: El distrito de ATE cuenta con reducidas zonas de areas verdes

Firmas: 5 / 5

Documentos Soporte: [Documentacion.pdf](#)

Comentarios: Apoyo total

Firmar

Escribe un comentario...

Comentar

Choose File

No file chosen

Subir Recurso

Enviar a Congreso

2. En el mongodb

```
▶ _id: ObjectId('6a22f4c2aaca5e65af2a9b18')
  id: "071116a7"
  titulo: "Aumento de Areas verdes en el distrito de ATE "
  contenido: "Capítulo: Disposiciones Generales
              - Artículo: El distrito de ATE cue..."
  firmas: 5
  estado: "Congelada"
  ▶ comentarios: Array (1)
  ▶ adjuntos: Array (1)
```

Resultado: El backend suma la 5ta firma. El Facade detecta la meta, cambia el estado a "Congelada" en MongoDB y dispara el imprime notificación interna en consola.

Aumento de Areas verdes en el distrito de ATE

Congelada

Capítulo: Disposiciones Generales

- Artículo: El distrito de ATE cuenta con reducidas zonas de areas verdes

Firmas:

Documentos:

Comentarios:

Code

No se puede firmar una iniciativa cerrada.

Aceptar

Firmar

Escribe un comentario...

Comentar

Choose File

No file chosen

Subir Recurso

Enviar a Congreso

4. Patrones de diseño

4.1 Facade

- **¿Qué hace?** La clase `SistemaVozCiudadanaFacade` es el cerebro central. Recibe las peticiones de las rutas de FastAPI y se encarga por detrás de conectar con MongoDB, llamar a los demás patrones y guardar los datos. Gracias a este patrón, tus rutas (routes.py) están limpias y no tienen idea de cómo funciona la base de datos ni la lógica de negocio.

```
class SistemaVozCiudadanaFacade:
    def __init__(self):
        # Conexión a Mongo
        self.uri = "mongodb+srv://erickslonga24_db_user:ltg9xCcFwERsCsJl@todolistcluster.hrygcsc.mongodb.net/?appName=TodoListCluster"
        self.client = MongoClient(self.uri)
        self.db = self.client["voz_ciudadana_db"]
        self.collection = self.db["iniciativas"]
        self.limite_firmas = 5
```

Conexion a MongoDB

4.2 Decorator

- **¿Qué hace?** Las clases `ComentarioDecorator` y `AdjuntoDecorator` toman una iniciativa básica y la "envuelven" para agregarle los textos de los comentarios o las rutas de los archivos físicos que sube el usuario (uploads/...). Esto evita tener que crear subclases rígidas

```

class ComentarioDecorator(IniciativaInteractuable):
    def __init__(self, iniciativa: IniciativaBase, comentario: str):
        self.iniciativa = iniciativa
        self.iniciativa._comentarios.append(comentario)
    def obtener_detalle(self) -> dict:
        return self.iniciativa.obtener_detalle()

class AdjuntoDecorator(IniciativaInteractuable):
    def __init__(self, iniciativa: IniciativaBase, adjunto: str):
        self.iniciativa = iniciativa
        self.iniciativa._adjuntos.append(adjunto)
    def obtener_detalle(self) -> dict:
        return self.iniciativa.obtener_detalle()

```

4.3 Proxy

- **¿Qué hace?** La clase `IniciativaProxy` se coloca frente a la iniciativa real como un "guardia de seguridad". Cuando el usuario intenta modificar el texto o enviar la iniciativa al Congreso, el Proxy revisa el estado actual. Si la iniciativa está "Congelada", bloquea la edición; si no está "Congelada", bloquea el envío.

```

class IniciativaProxy:
    def __init__(self, iniciativa: IniciativaBase):
        self.iniciativa = iniciativa

    def modificar_contenido(self, nuevo_texto: str):
        if self.iniciativa.estado.nombre in ["Congelada", "Enviada"]:
            raise PermissionError("Acción denegada: La iniciativa está bloqueada o cerrada.")
        nuevo_capitulo = Capitulo("Capítulo Único Modificado")
        nuevo_capitulo.agregar(Articulo(nuevo_texto))
        self.iniciativa.contenido_jerarquico = nuevo_capitulo
        return True

    def enviar_congreso(self):
        if self.iniciativa.estado.nombre != "Congelada":
            raise PermissionError("Acción denegada: Solo se envían iniciativas congeladas.")
        self.iniciativa.estado = EstadoFactory.get_estado("Enviada")
        return True

```

4.4. Composite

- **¿Qué hace?** Modela la estructura legal de la iniciativa. Un `Capitulo` contiene varios hijos (`Articulo`). Gracias al Composite, cuando el sistema necesita mostrar el contenido en la tarjeta, simplemente llama a la función `renderizar()` y el árbol entero se dibuja a sí mismo, combinando capítulos y artículos de forma jerárquica.

```

class ComponenteIniciativa:
    def renderizar(self) -> str:
        pass

class Artículo(ComponenteIniciativa):
    def __init__(self, texto: str):
        self.texto = texto
    def renderizar(self) -> str:
        return f"Artículo: {self.texto}"

class Capitulo(ComponenteIniciativa):
    def __init__(self, titulo: str):
        self.titulo = titulo
        self.hijos = []
    def agregar(self, componente: ComponenteIniciativa):
        self.hijos.append(componente)
    def renderizar(self) -> str:
        contenido = f"Capítulo: {self.titulo}\n"
        for hijo in self.hijos:
            contenido += f"    {hijo.renderizar()}\n"
        return contenido

```

4.5 Flyweight

- **¿Qué hace?** La clase EstadoFactory gestiona los estados de las iniciativas ("Activa", "Congelada", "Enviada"). En lugar de que tus 1,000 iniciativas en la base de datos creen 1,000 objetos de estado en la memoria del servidor, el Flyweight asegura que solo exista una única instancia de cada estado en todo el sistema, y todas las iniciativas simplemente apuntan a ella.

```

class EstadoIniciativaFlyweight:
    def __init__(self, nombre: str):
        self.nombre = nombre

class EstadoFactory:
    _estados = {}

    @staticmethod
    def get_estado(nombre: str) -> EstadoIniciativaFlyweight:
        if nombre not in EstadoFactory._estados:
            EstadoFactory._estados[nombre] = EstadoIniciativaFlyweight(nombre)
        return EstadoFactory._estados[nombre]

```